



FPT POLYTECHNIC



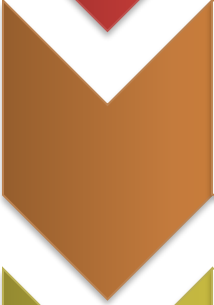
Conceive Design Implement Operate

BÀI 6: CÔNG CỤ TRỰC QUAN HÓA DỮ LIỆU

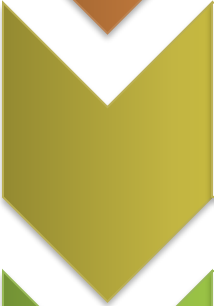
www.poly.edu.vn



So sánh Công cụ



BI Dashboards



Python Core



D3.js Web



Vai trò của Trực quan hóa trong AI

- Khám phá dữ liệu (EDA): Hiểu phân phối và mối liên hệ trước khi huấn luyện mô hình
- Đánh giá mô hình: Trực quan hóa sai số, ma trận nhầm lẫn (Confusion Matrix)
- Giải thích AI (XAI): Biến "hộp đen" thuật toán thành thông tin có thể hiểu được
- Truyền tải tri thức: Thuyết phục các bên liên quan bằng câu chuyện dữ liệu



1. So SÁNH CÔNG CỤ

Thị trường Công cụ Trực quan hóa

Theo Gartner 2024, thị trường chia thành hai nhóm chính

1. Nền tảng BI (No-Code/Low-Code): Tập trung vào doanh nghiệp, báo cáo định kỳ và quản trị (Power BI, Tableau)
2. Công cụ lập trình (Code-Centric): Tập trung vào tính tùy biến, nghiên cứu khoa học và tích hợp sản phẩm (Python, D3.js)

Thị trường Công cụ Trực quan hóa

Theo Gartner 2024, thị trường chia thành hai nhóm chính

1. Nền tảng BI (No-Code/Low-Code): Tập trung vào doanh nghiệp, báo cáo định kỳ và quản trị (Power BI, Tableau)
2. Công cụ lập trình (Code-Centric): Tập trung vào tính tùy biến, nghiên cứu khoa học và tích hợp sản phẩm (Python, D3.js)

Đặc tính Nhóm BI Tools

Ưu điểm

- Triển khai cực nhanh cho dữ liệu doanh nghiệp lớn
- Khả năng kéo thả mạnh mẽ
- Hỗ trợ nhiều nguồn dữ liệu (SQL, Cloud, Excel)

Nhược điểm

- Chi phí bản quyền cao
- Khó khăn khi muốn thực hiện các biểu đồ phi truyền thống hoặc tích hợp sâu vào mã nguồn AI phức tạp

Đặc tính Nhóm Lập trình

Ưu điểm

- Miễn phí, mã nguồn mở
- Tự động hóa hoàn toàn quy trình xử lý dữ liệu
- Phù hợp môi trường AI/ML

Nhược điểm

- Cần kỹ năng lập trình cao
- Tốn thời gian để thiết kế được giao diện dashboard so với kéo thả

Ma trận So sánh Công cụ

Tiêu chí	Tableau	Power BI	Python (Seaborn)	D3.js
Dành cho	Data Analyst chuyên sâu	Business User	Data Scientist / AI Eng	Web Developer
Độ phức tạp	Trung bình	Thấp (Kéo thả)	Trung bình (Coding)	Rất cao
Tùy biến	Cao	Trung bình	Rất cao	Vô hạn
Tương tác	Mạnh mẽ	Cơ bản (Slicer)	Tĩnh (hoặc Interactive)	Cực kỳ linh hoạt



2. TABLEAU / POWER BI

Quy trình 4 bước xây dựng Dashboard

Kết nối
(Connect)

Xử lý
(Transform)

Thiết kế
(Visualize)

Chia sẻ
(Publish)



Import

Power Query

Kéo thả các

Web

CSV/SQL/Cloud

Data Prep

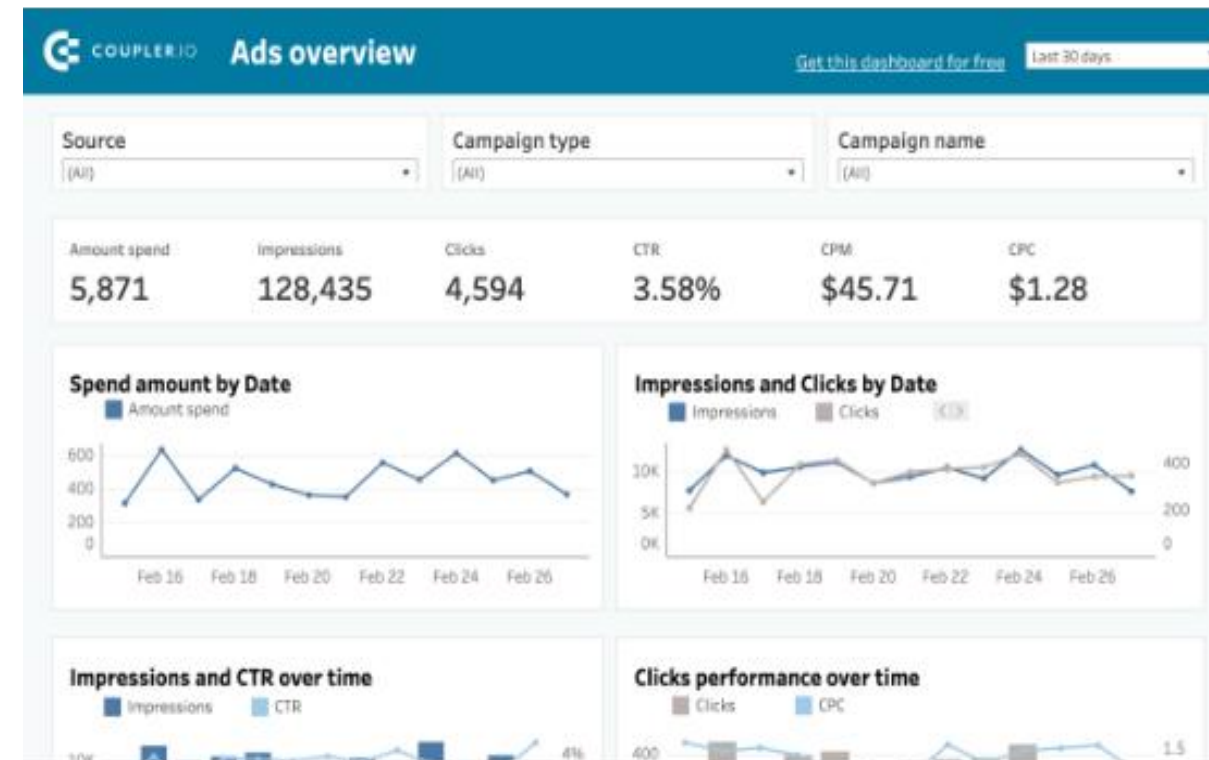
biểu đồ

Mobile App

Tableau: Triết lý “Show me”

Giúp gợi ý biểu đồ thông minh

- Khám phá nhanh: Khả năng xử lý hàng triệu bản ghi trong tích tắc
- Thẩm mỹ: Hệ thống màu sắc và bố cục tinh tế
- Tính tương tác: Action filters cực kỳ mạnh mẽ



Power BI: Hệ sinh thái Microsoft

Lựa chọn hàng đầu cho doanh nghiệp đang sử dụng Windows/Office 365

- DAX (Data Analysis Expressions): Ngôn ngữ tính toán mạnh mẽ tương tự Excel
- Tích hợp: Kết nối hoàn hảo với Azure, Excel, SQL Server
- Giá cả: Gói Pro rất cạnh tranh cho doanh nghiệp vừa và nhỏ

Nguyên tắc Thiết kế Dashboard

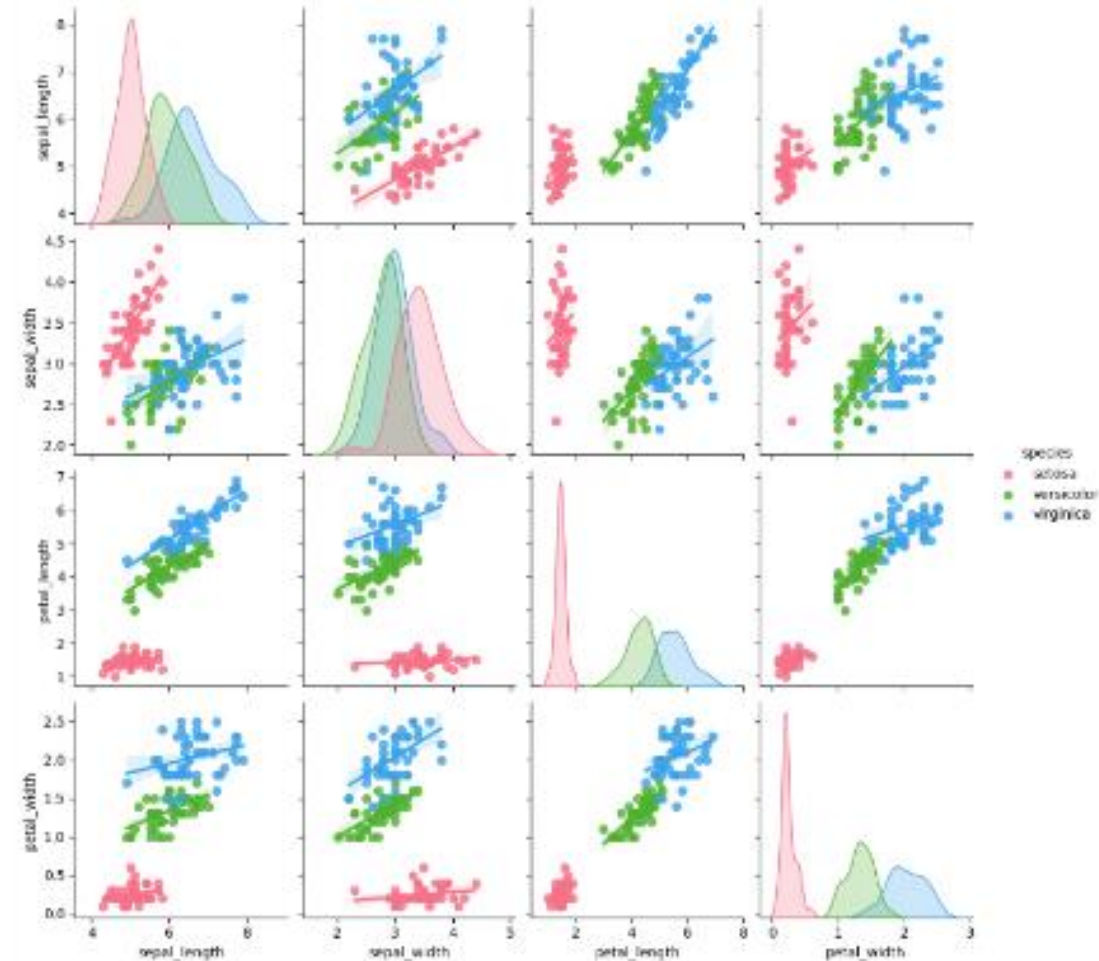
- **Phân cấp thông tin:** Đưa các chỉ số KPI quan trọng nhất lên góc trên bên trái
- **Hạn chế biểu đồ:** Không quá 5-6 biểu đồ trên một màn hình để tránh quá tải thị giác
- **Màu sắc có ý nghĩa:** Sử dụng màu để phân biệt dữ liệu, không phải để trang trí
- **Tăng cường tương tác:** Cho phép người dùng lọc dữ liệu để tự khám phá câu chuyện của họ



3. PYTHON VIZ

Tại sao chọn Python cho AI Viz?

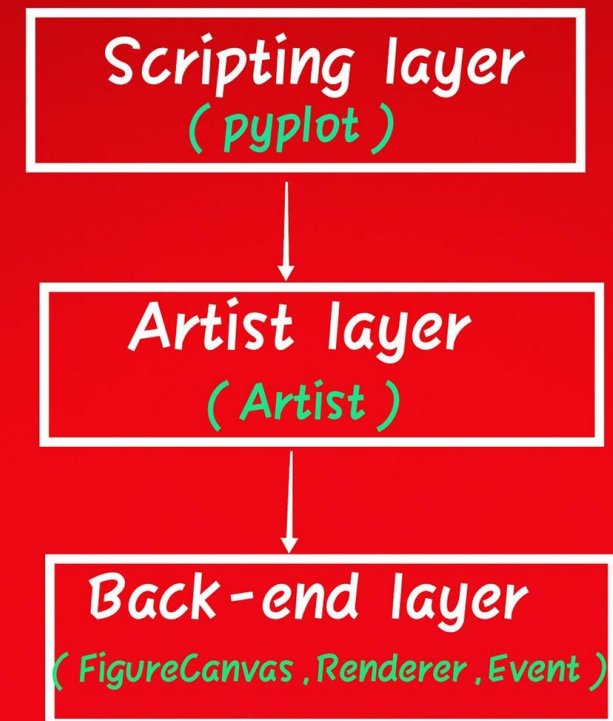
- **Reproducibility:** Lưu trữ mã nguồn giúp tái lập biểu đồ dễ dàng
- **Tích hợp:** Vẽ trực tiếp từ mảng Numpy hoặc DataFrame của Pandas
- **ML Pipeline:** Tự động xuất biểu đồ trong quá trình train mô hình



Kiến trúc Matplotlib

- **Scripting Layer:** Tầng pyplot (`plt`) - Đơn giản nhất để vẽ biểu đồ nhanh
- **Artist Layer:** Tùy chỉnh từng chi tiết (nhãn, trục, lưới)
- **Backend Layer:** Tầng xử lý xuất file (PNG, PDF, SVG) hoặc hiển thị lên màn hình

Architecture of Matplotlib



Matplotlib: Biểu đồ cơ sở

- **plt.plot()**: Biểu đồ đường cho chuỗi thời gian
- **plt.bar()**: Biểu đồ cột cho so sánh danh mục
- **plt.scatter()**: Biểu đồ tán xạ xem tương quan
- **plt.hist()**: Biểu đồ tần suất xem phân phối

Seaborn: Thống kê và Thẩm mỹ

- **Cú pháp ngắn:** Vẽ biểu đồ phức tạp chỉ với 1 dòng code
 - **Themes mặc định:** Giao diện chuyên nghiệp ngay lập tức
 - **Hỗ trợ Pandas:** Hoạt động hoàn hảo với DataFrame
- > *Matplotlib cung cấp "khung xương", Seaborn cung cấp "vẻ đẹp"*

Phân tích Phân phối với Seaborn

- **sns.displot()**: Phân phối linh hoạt (Histogram + KDE)
- **sns.boxplot()**: Xem độ lệch và các giá trị ngoại lai (outliers)
- **sns.violinplot()**: Kết hợp boxplot và mật độ phân phối

Ma trận Tương quan & Heatmaps

Sử dụng **`sns.heatmap(df.corr(), annot=True)`** để:

- Quan sát mọi mối quan hệ cặp giữa các biến số
- Nhận diện các cụm (clusters) dữ liệu một cách trực quan
- Tìm kiếm các ranh giới phân tách lớp (Class separability)

Tùy biến Nâng cao: Grids

- FacetGrid: Chia nhỏ biểu đồ theo các biến phân loại (categories)
- JointGrid: Kết hợp biểu đồ tán xạ và biểu đồ mật độ biên
- Tùy chỉnh rcParam để đặt chuẩn thẩm mỹ cho toàn bộ báo cáo

Trực quan hóa cho Machine Learning

Giai đoạn	Loại biểu đồ ưu tiên	Mục đích
Pre-processing	Boxplot, Histogram, Pairplot	Làm sạch nhiễu, hiểu đặc trưng
Training	Line Plot (Loss/Accuracy)	Theo dõi hiện tượng Overfitting
Evaluation	Confusion Matrix, ROC Curve	Đánh giá độ chính xác thực tế
Deployment	Feature Importance Bar Chart	Giải thích tại sao AI ra quyết định

Một số lưu ý trình bày

- Xóa các đường lưới không cần thiết, nhãn thừa
- Luôn có tiêu đề, tên trục và đơn vị đo lường
- Đảm bảo mọi người đều đọc được
- Ưu tiên định dạng `.svg` hoặc `.pdf` cho báo cáo chất lượng cao



4. D3.JS CHO ỨNG DỤNG WEB

Giới thiệu

D3.js không chỉ là một thư viện đồ thị có sẵn, mà là một công cụ thao tác dữ liệu

- **Data Binding:** Ràng buộc dữ liệu vào các thành phần HTML/SVG
- **Thao tác DOM:** Sử dụng ``select``, ``enter``, ``update``, ``exit``
- **Transitions:** Hiệu ứng chuyển động mượt mà

Thành phần cốt lõi của D3.js

- **SVG (Scalable Vector Graphics):** Vẽ các hình khối (circle, rect, path) không bị vỡ ảnh khi phóng to
- **Scales & Axes:** Chuyển đổi giá trị dữ liệu sang tọa độ điểm ảnh (pixel) trên màn hình
- **Interactivity:** Hỗ trợ zoom, drag, hover và các sự kiện click

Ví dụ: Scatter Plot cho Accuracy và Training Time

- Import thư viện D3 v7
- Tạo 1 thẻ <svg> — đây là "canvas" để vẽ
- Mọi thứ trong D3 thực chất là thao tác DOM với SVG

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>D3 Scatter Plot - AI Models</title>
  <script src="https://d3js.org/d3.v7.min.js"></script>
</head>
<body>

  <h2>So sánh mô hình AI</h2>
  <svg width="600" height="400"></svg>

</body>
</html>
```

Ví dụ: Scatter Plot cho Accuracy và Training Time

Thêm thẻ <script>, bên trong có:

- Mảng object
- Mỗi object = 1 mô hình
- D3 sẽ bind từng object vào 1 phần tử SVG

```
...  
<svg width="600" height="400"></svg>  
  
<script>  
const data = [  
  {model: "SVM", accuracy: 88, time: 12},  
  {model: "Random Forest", accuracy: 92, time: 25},  
  {model: "KNN", accuracy: 85, time: 5},  
  {model: "Neural Network", accuracy: 95, time: 40},  
  {model: "Naive Bayes", accuracy: 80, time: 3}  
];  
</script>  
  
</body>  
</html>
```

Ví dụ: Scatter Plot cho Accuracy và Training Time

- `d3.select("svg")` → chọn phần tử DOM
- Thiết lập kích thước
- Margin giúp chừa chỗ cho trục

```
...  
<script>  
...  
  
const svg = d3.select("svg");  
  
const width = 600;  
const height = 400;  
  
const margin = {top: 20, right: 30, bottom: 40, left:  
50};  
</script>  
...
```

Ví dụ: Scatter Plot cho Accuracy và Training Time

Tạo scale cho trục X và Y

- domain() = không gian dữ liệu
- range() = không gian pixel

```
...  
<script>  
...  
  
const x = d3.scaleLinear()  
  .domain([0, d3.max(data, d => d.time)])  
  .range([margin.left, width - margin.right]);  
  
const y = d3.scaleLinear()  
  .domain([70, 100])  
  .range([height - margin.bottom, margin.top]);  
</script>  
...
```

Ví dụ: Scatter Plot cho Accuracy và Training Time

Vẽ trục tọa độ

- `append("g")` = group SVG
- `call(axis)` = D3 vẽ trục dựa vào scale
- `transform` để đặt đúng vị trí

```
...  
<script>  
...  
  
svg.append("g")  
  .attr("transform", `translate(0,${height -  
margin.bottom})`)  
  .call(d3.axisBottom(x));  
  
svg.append("g")  
  .attr("transform", `translate(${margin.left},0)`)  
  .call(d3.axisLeft(y));  
</script>  
...
```


Ví dụ: Scatter Plot cho Accuracy và Training Time

Bind dữ liệu và vẽ điểm:

- `selectAll("circle")`: Chọn tất cả circle
- `.data(data)`: Bind mảng dữ liệu
- `.enter()`: Tạo placeholder cho mỗi phần tử dữ liệu mới
- `.append("circle")`: Tạo 1 circle cho mỗi object



...

`<script>`

...

```
svg.selectAll("circle")  
  .data(data)  
  .enter()  
  .append("circle")  
  .attr("cx", d => x(d.time))  
  .attr("cy", d => y(d.accuracy))  
  .attr("r", 6)  
  .attr("fill", "steelblue");  
</script>
```

...

Ví dụ: Scatter Plot cho Accuracy và Training Time

Thêm label tên mô hình



...

```
<script>
```

...

```
svg.selectAll("text.label")  
  .data(data)  
  .enter()  
  .append("text")  
  .attr("x", d => x(d.time) + 8)  
  .attr("y", d => y(d.accuracy))  
  .text(d => d.model)  
  .style("font-size", "12px");  
</script>
```

...

Ví dụ: Scatter Plot cho Accuracy và Training Time

So sánh mô hình AI





FPT POLYTECHNIC

THANK YOU

